

A MINOR PROJECT REPORT ON

**CNN IMAGE CLASSIFICATION USING
THE CIFAR-10 DATASET**

Submitted by
Sudiksha Chauhan
Roll Number: 2400290120251

DEPARTMENT OF COMPUTER SCIENCE

KIET DEEMED TO BE UNIVERSITY

Academic Year: 2025 – 2026

1. PROBLEM STATEMENT

Image classification is one of the foundational tasks in computer vision and machine learning. It involves assigning a class label to an image based on its visual content. While humans can distinguish between a photograph of a dog and a ship almost effortlessly, teaching a computer to do the same requires a model that can learn and extract meaningful visual features directly from raw pixel data.

Traditional machine learning approaches, which rely on manually engineered features, tend to perform poorly on image data because raw images are high-dimensional and the relevant features (edges, textures, shapes, and object parts) are not easily captured by hand-crafted rules. Deep learning, and Convolutional Neural Networks in particular, have transformed this area by allowing the model to learn hierarchical feature representations directly from the pixel values during training.

This project addresses the problem of classifying 32×32 colour images from the CIFAR-10 dataset into one of ten object categories. The dataset is well-known in the research community and is considered a benchmark for evaluating image classification models, particularly because the small image resolution (32×32 pixels) makes the task challenging: fine visual details are lost at this resolution, making it difficult for a model to distinguish between visually similar classes such as Cat and Dog. The goal of this project is to build, train, and evaluate a CNN that learns to classify these images accurately.

2. DATASET DESCRIPTION

The CIFAR-10 dataset (Canadian Institute for Advanced Research – 10 classes) is a widely used benchmark dataset for image classification. It was originally collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. In this project it was loaded directly from the TensorFlow/Keras API using `cifar10.load_data()`, which returns the dataset pre-split into training and testing subsets.

2.1 Dataset Statistics

Property	Value
Total Images	60,000
Training Images	50,000
Test Images	10,000
Image Height	32 pixels
Image Width	32 pixels
Colour Channels	3 (Red, Green, Blue)
Number of Classes	10
Images per Class (Train)	5,000
Images per Class (Test)	1,000
Pixel Value Range (Raw)	0 – 255 (uint8)
Pixel Value Range (Normalized)	0.0 – 1.0 (float32)

Table 2.1: CIFAR-10 Dataset Statistics

2.2 Class Labels

The dataset contains images belonging to ten mutually exclusive classes. Each image has exactly one label. The ten classes and their corresponding integer labels (as used internally by the model) are:

Class Index	Class Name	Class Index	Class Name
0	Airplane	5	Dog
1	Automobile	6	Frog
2	Bird	7	Horse
3	Cat	8	Ship

4	Deer	9	Truck
---	------	---	-------

Table 2.2: CIFAR-10 Class Labels

3. DATA PREPROCESSING STEPS

The CIFAR-10 dataset, as loaded from the Keras API, is already structured and split. The preprocessing applied in this project consists of two straightforward but essential steps.

3.1 Loading the Dataset

The dataset was loaded using `cifar10.load_data()`, which returned two tuples: `(X_train, y_train)` and `(X_test, y_test)`. The training data `X_train` has the shape `(50000, 32, 32, 3)`, meaning 50,000 images each with a height of 32 pixels, a width of 32 pixels, and 3 colour channels (RGB). The training labels `y_train` have the shape `(50000, 1)`, where each entry is an integer from 0 to 9 corresponding to one of the ten class categories. The test data `X_test` has the shape `(10000, 32, 32, 3)` and the test labels `y_test` the shape `(10000, 1)`.

3.2 Pixel Normalization

In their raw form, the pixel values in CIFAR-10 images are unsigned 8-bit integers in the range 0 to 255. Before passing this data to a neural network, both `X_train` and `X_test` were first cast to 32-bit floating point (`float32`) and then divided by 255.0. This rescales every pixel value to the range `[0.0, 1.0]`, which was confirmed in the notebook output (Maximum Pixel Value: 1.0, Minimum Pixel Value: 0.0).

Normalization is a standard and important preprocessing step in deep learning. Neural networks, and in particular the gradient descent optimization used during training, converge more reliably and more quickly when input features are in a consistent, bounded range. Without normalization, the large variance in raw pixel values (0–255) can cause the network's weights to update in large, unstable steps during training.

No further preprocessing (such as data augmentation, zero-centering, or PCA whitening) was applied in this project. The labels were used in their raw integer form, which is compatible with the sparse categorical cross-entropy loss function used during compilation.

4. EXPLORATORY DATA ANALYSIS

Before building the model, a visual inspection of the training dataset was carried out to understand the nature and appearance of the images in the dataset. Since CIFAR-10 is a well-defined benchmark, no statistical EDA (such as class distribution charts or pixel histograms) was performed in the notebook; the primary exploratory step was a direct visual display of sample training images.

4.1 Sample Training Images



Figure 4.1: Sample training images from the CIFAR-10 dataset (one per class)

Ten sample images from the training set were displayed in a 2×5 grid, one from each of the first ten records in the training data. The visualization reveals several important characteristics of the CIFAR-10 dataset. First, the images are very small (32×32 pixels), which means that fine object details are lost and images appear somewhat blurry or pixelated when displayed at a larger scale. Second, the classes span a wide variety of objects, from natural living things such as birds, deer, horses, and frogs, to man-made objects such as airplanes, automobiles, ships, and trucks. Third, even from this small sample, it is apparent that certain classes (particularly Cat and Dog, and similarly Automobile and Truck) are visually similar to each other, which is one of the main reasons that CIFAR-10 is considered a challenging dataset for classification.

The images confirmed that the dataset was loaded correctly and that the `class_names` list maps accurately to the label indices, as the displayed label for each image matched the object visible in the image.

5. MODEL USED

This project uses a Convolutional Neural Network (CNN) implemented using TensorFlow's Keras Sequential API. CNNs are a class of deep learning models specifically designed for processing data that has a grid-like spatial structure, such as images. They have become the standard approach for image classification tasks because they can automatically learn spatial hierarchies of features: lower layers detect simple features like edges and colour gradients, while deeper layers progressively combine these into more complex patterns such as textures, object parts, and eventually whole objects.

The key advantage of a CNN over a traditional fully connected (Dense) neural network for image data is the use of convolutional filters, which share weights across the spatial extent of the image. This weight sharing dramatically reduces the number of learnable parameters compared to a fully connected network and also makes the model naturally invariant to small translations of the object within the image (a cat shifted slightly to the left is still recognized as a cat).

In this project, a relatively compact CNN was built from scratch with 167,562 total trainable parameters. This size was chosen to be appropriate for the 32×32 input resolution of CIFAR-10 and to keep training time manageable without a GPU-accelerated environment.

6. CNN ARCHITECTURE

The model was built using the `tf.keras.Sequential` API, which stacks layers in a linear order from input to output. The complete architecture is described layer by layer below.

6.1 Layer-by-Layer Description

Layer 1: Conv2D (32 filters, 3×3 kernel, ReLU activation) — Output: (None, 30, 30, 32)

The first layer is a 2D convolutional layer with 32 filters (also called feature maps) and a 3×3 kernel size. A convolutional filter is a small learnable matrix that slides across the input image and computes a dot product with each 3×3 patch it covers. Because 32 filters are used, this layer produces 32 separate feature maps, each capturing a different low-level visual feature such as horizontal edges, vertical edges, or colour gradients. The input shape is specified as (32, 32, 3) matching the CIFAR-10 image dimensions. After applying the 3×3 filter with no padding (valid padding), the spatial dimensions reduce from 32×32 to 30×30, giving an output shape of (None, 30, 30, 32). Each filter has $3 \times 3 \times 3 = 27$ weights plus 1 bias, so with 32 filters the total learnable parameters in this layer are $(27 + 1) \times 32 = 896$.

The ReLU (Rectified Linear Unit) activation function is applied element-wise after the convolution. ReLU replaces any negative value with zero and keeps positive values unchanged: $f(x) = \max(0, x)$. It introduces non-linearity into the network, which is essential because without a non-linear activation, stacking multiple layers would be equivalent to a single linear transformation. ReLU is preferred over older activation functions like sigmoid or tanh in CNNs because it does not suffer from the vanishing gradient problem and is computationally very efficient.

Layer 2: MaxPooling2D (2×2 pool size) — Output: (None, 15, 15, 32)

A MaxPooling2D layer with a 2×2 pool size and default stride of 2 is applied after the first convolutional layer. This layer divides each feature map into non-overlapping 2×2 windows and keeps only the maximum value from each window. This halves the spatial dimensions of the feature maps from 30×30 to 15×15, reducing the number of values that need to be processed by subsequent layers. MaxPooling serves two purposes: it reduces computational cost, and it makes the feature representations more robust to small spatial shifts in the input, since a feature detected

slightly to the left or right of the exact position will still produce the same maximum value in its pooling window. This layer has no learnable parameters.

Layer 3: Conv2D (64 filters, 3×3 kernel, ReLU activation) — Output: (None, 13, 13, 64)

The second convolutional layer has 64 filters with the same 3×3 kernel size. Because its input is the 15×15×32 output of the previous pooling layer, each filter now operates over $3 \times 3 \times 32 = 288$ input values plus a bias, and with 64 filters the total parameters are $(288 + 1) \times 64 = 18,496$. After applying the 3×3 filters without padding, the spatial dimensions reduce from 15×15 to 13×13. By using twice as many filters as the first convolutional layer, this layer can learn more complex, higher-level feature combinations built from the simpler features extracted by the first layer. ReLU activation is applied here as well.

Layer 4: MaxPooling2D (2×2 pool size) — Output: (None, 6, 6, 64)

A second MaxPooling2D layer with a 2×2 pool size is applied after the second convolutional layer. This further halves the spatial dimensions from 13×13 to 6×6 (with one row/column dropped due to the odd input size), producing an output of shape (None, 6, 6, 64). As before, this layer has no trainable parameters. After two rounds of convolution and pooling, the 32×32 input images have been transformed into compact 6×6 feature maps that represent high-level visual features of the image.

Layer 5: Flatten — Output: (None, 2304)

The Flatten layer converts the three-dimensional tensor of shape (6, 6, 64) into a one-dimensional vector of length $6 \times 6 \times 64 = 2,304$. This step is necessary because the subsequent Dense (fully connected) layers require a one-dimensional input. No computation or learning happens in this layer; it is purely a reshaping operation with no trainable parameters.

Layer 6: Dense (64 units, ReLU activation) — Output: (None, 64)

The first Dense layer is a fully connected layer with 64 neurons. Each of the 2,304 inputs from the flattened vector is connected to all 64 neurons, giving $2,304 \times 64 = 147,456$ weight connections plus 64 biases, for a total of 147,520 trainable parameters. This layer combines the spatial features extracted by the convolutional blocks into global representations of the image, allowing the network to form high-level decisions about class membership. ReLU activation is used here as well to maintain non-linearity.

Layer 7: Dense (10 units, Softmax activation) — Output: (None, 10)

The final Dense output layer has 10 neurons, one for each of the 10 CIFAR-10 classes. It has $64 \times 10 = 640$ weight connections plus 10 biases, giving 650 trainable parameters. The Softmax activation function converts the raw output scores (logits) into a probability distribution across the 10 classes: each output value is in the range (0, 1) and all ten outputs sum to exactly 1. The predicted class for a given image is the one with the highest probability in this output vector.

6.2 Model Summary Table

Table 6.1 presents the complete model summary as produced by `model.summary()` in the notebook.

Layer (Type)	Output Shape	Parameters
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 64)	147,520
dense_1 (Dense)	(None, 10)	650
Total Trainable Parameters		167,562

Table 6.1: CNN Model Summary

7. TRAINING PROCESS

After building the model architecture, the model was compiled and trained using the following configuration.

7.1 Model Compilation

The model was compiled with three settings. The optimizer used was Adam (Adaptive Moment Estimation), which is an adaptive learning rate gradient descent algorithm that adjusts the learning rate individually for each weight during training. Adam is widely preferred over standard SGD in deep learning because it converges faster and is less sensitive to the initial learning rate. The loss function used was `sparse_categorical_crossentropy`, which is appropriate here because the class labels are integers (0–9) rather than one-hot encoded vectors. This loss function measures the cross-entropy between the true class label and the predicted probability distribution produced by the Softmax output layer. The evaluation metric tracked during training and validation was accuracy.

7.2 Training Configuration

Parameter	Value
Training samples used	40,000 (80% of 50,000; <code>validation_split = 0.2</code>)
Validation samples used	10,000 (20% of 50,000; internal split)
Epochs	10
Batch size	32
Optimizer	Adam
Loss Function	Sparse Categorical Cross-Entropy
Evaluation Metric	Accuracy

Table 7.1: Model Training Configuration

The `validation_split=0.2` parameter instructs Keras to reserve the last 20% of the training data (10,000 images) as a validation set. The model never trains on validation images; they are only used to compute the validation accuracy and validation loss at the end of each epoch. This provides an ongoing estimate of how well the model generalizes to data it has not been trained on, without touching the separate test set.

7.3 Epoch-by-Epoch Training History

Table 7.2 presents the training accuracy, training loss, validation accuracy, and validation loss recorded at the end of each of the 10 training epochs.

Epoch	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss
1	45.34%	1.5151	54.14%	1.2974
2	58.74%	1.1715	60.76%	1.1101
3	63.65%	1.0400	63.54%	1.0267
4	66.99%	0.9496	66.56%	0.9785
5	69.54%	0.8788	66.58%	0.9663
6	71.76%	0.8137	67.39%	0.9553
7	73.53%	0.7626	68.73%	0.9356
8	75.09%	0.7160	68.97%	0.9204
9	76.80%	0.6679	68.97%	0.9222
10	78.45%	0.6204	69.42%	0.9373

Table 7.2: Epoch-by-epoch training and validation metrics

8. EVALUATION METRICS AND RESULTS

8.1 Training and Validation Accuracy Curve

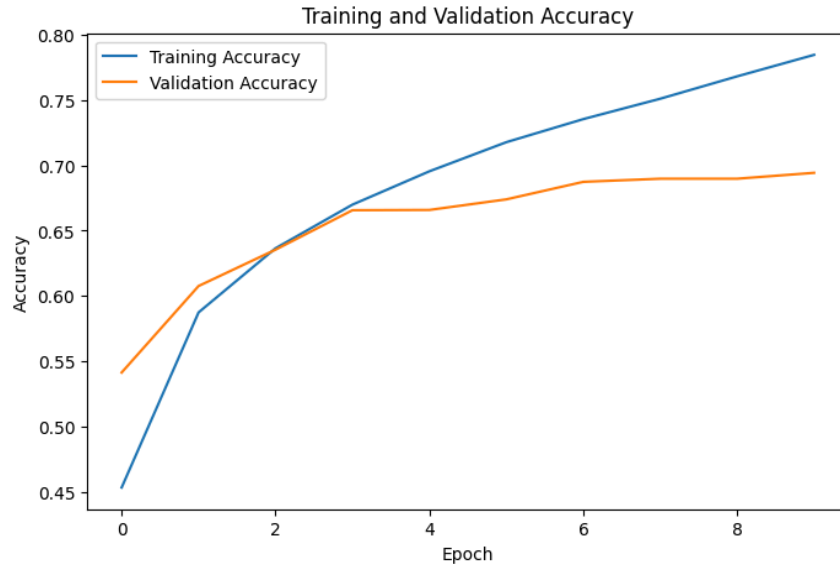


Figure 8.1: Training and Validation Accuracy over 10 Epochs

The accuracy curve shows that both training and validation accuracy improve consistently across all 10 epochs. Training accuracy rises from 45.34% in epoch 1 to 78.45% by epoch 10. Validation accuracy follows a similar trajectory, improving from 54.14% to 69.42%, though it plateaus and grows more slowly from epoch 5 onwards while training accuracy continues to increase. The growing gap between training and validation accuracy in the later epochs (roughly 9 percentage points by epoch 10) indicates the onset of overfitting: the model is beginning to memorize specific patterns in the training data that do not fully generalize to unseen images. Despite this, the validation accuracy remains stable rather than declining, which suggests that the overfitting is mild at this stage.

8.2 Training and Validation Loss Curve

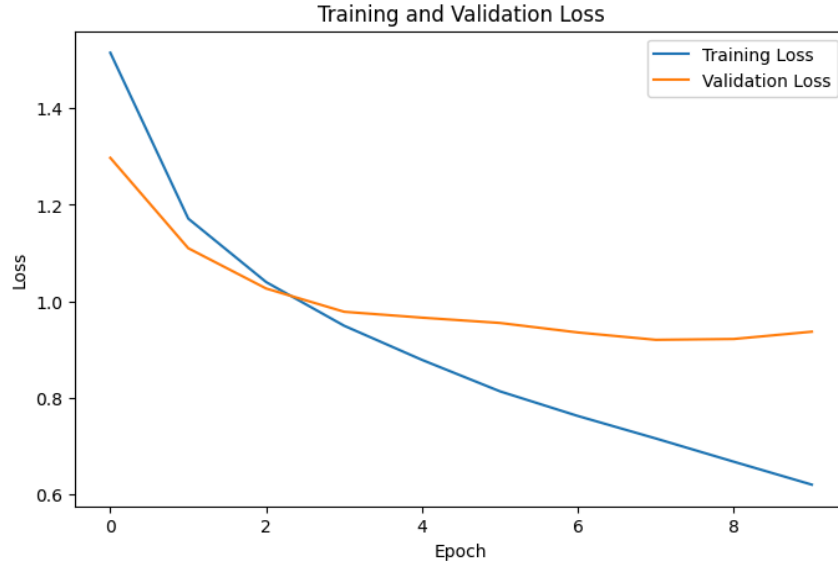


Figure 8.2: Training and Validation Loss over 10 Epochs

The loss curve reinforces the observations from the accuracy curve. Training loss decreases steadily from 1.5151 to 0.6204 across all 10 epochs, reflecting that the model is continuously improving its predictions on the training data. Validation loss also decreases from epoch 1 through to epoch 8 (reaching its lowest value of 0.9204 at epoch 8), but then shows a slight increase at epochs 9 and 10 (0.9222 and 0.9373 respectively). This divergence between training loss (still falling) and validation loss (levelling off and starting to rise slightly) is a clear indicator of mild overfitting in the final epochs. In a more complete implementation, techniques such as dropout or early stopping based on validation loss could be used to counteract this.

8.3 Test Set Evaluation

After training was complete, the model was evaluated on the held-out test set of 10,000 images that were never seen during training or validation. The results were:

Metric	Value
Test Accuracy	69.13%
Test Loss	0.9537

Table 8.1: Final Test Set Evaluation Results

The test accuracy of 69.13% is very close to the final validation accuracy of 69.42%, which is reassuring because it confirms that the validation accuracy was a reliable estimate of

generalization performance throughout training. The slight difference (0.29 percentage points) is attributable to natural variation between the validation and test sets.

8.4 Sample Predictions

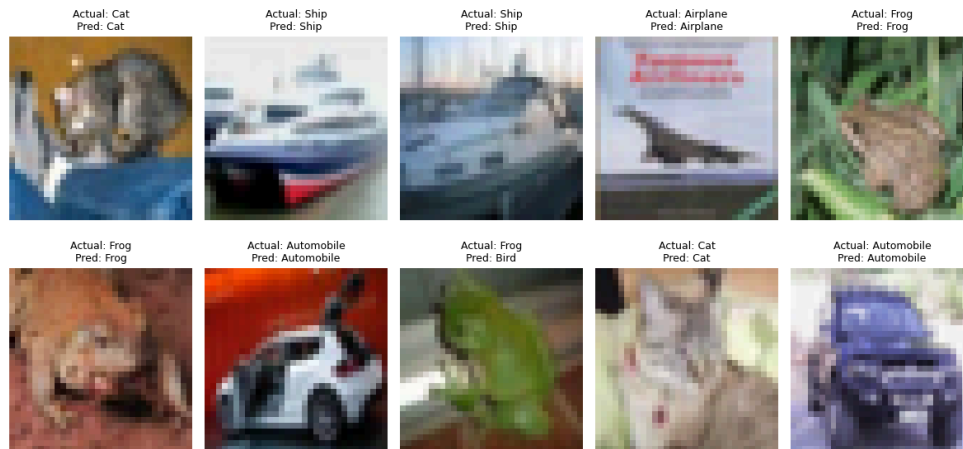


Figure 8.3: Sample test images with actual and predicted labels

Ten sample images from the test set were displayed with their actual and predicted labels to provide a qualitative view of the model's predictions. The text output from the notebook records the following individual results:

Image	Actual Class	Predicted Class	Correct?	
1	Cat	Cat	✓ Yes	
2	Ship	Ship	✓ Yes	
3	Ship	Ship	✓ Yes	
4	Airplane	Airplane	✓ Yes	
5	Frog	Frog	✓ Yes	
6	Frog	Frog	✓ Yes	
7	Automobile	Automobile	✓ Yes	
8	Frog	Bird	✗ No	
9	Cat	Cat	✓ Yes	
10	Automobile	Automobile	✓ Yes	

Table 8.2: Sample prediction results on the first 10 test images

Nine out of the ten displayed predictions are correct. The single misclassification (Image 8, Frog predicted as Bird) is understandable given that both classes can share similar colour schemes and body shapes in small 32×32 images.

8.5 Confusion Matrix

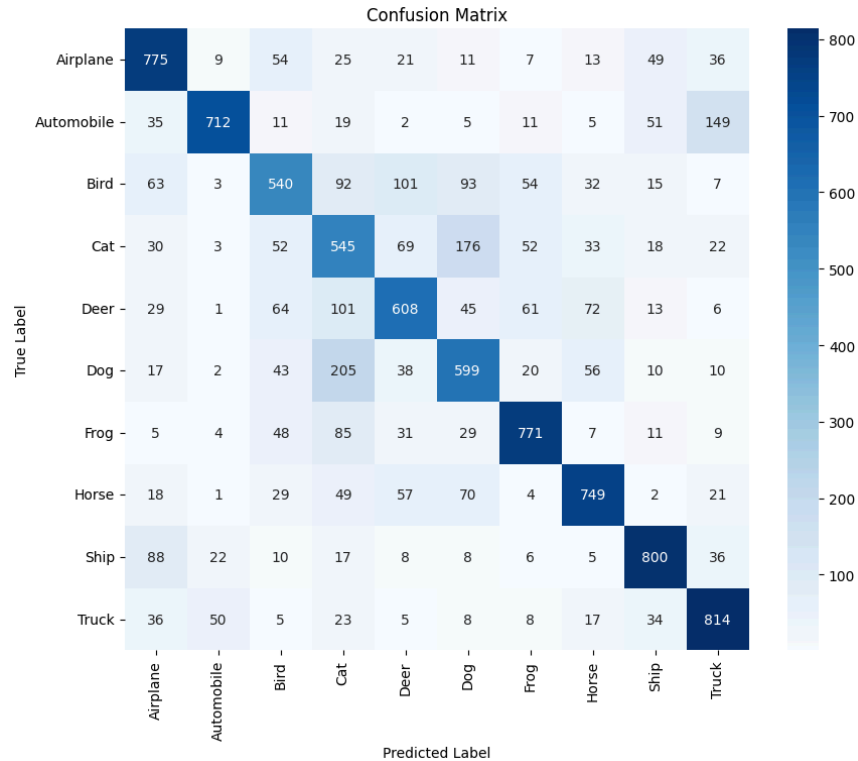


Figure 8.4: Confusion Matrix on the full 10,000-image test set

The confusion matrix presents a 10×10 grid where row i represents the true class and column j represents the predicted class. The diagonal values (top-left to bottom-right) represent the number of correctly classified images per class, while the off-diagonal values represent misclassifications. Key observations from the confusion matrix are:

Automobile is the strongest-performing class by precision (0.88), with most Automobile images correctly classified. The main source of confusion for this class is that some Truck images are predicted as Automobile and vice versa, which is expected given the visual similarity between these two vehicle types.

Cat has the lowest precision (0.47), meaning that nearly half of all images predicted as Cat actually belong to a different class. The most common confusion is between Cat and Dog, two

classes that look very similar at 32×32 resolution due to similar fur textures, body shapes, and poses.

Ship and Airplane both perform reasonably well (precisions of 0.80 and 0.71 respectively), likely because these classes have distinctive visual structures that are preserved even at low resolution.

Frog and Horse are also relatively well-classified (precisions of 0.78 and 0.76 respectively), with occasional confusion between visually similar natural classes such as Deer and Horse.

8.6 Per-Class Classification Report

The sklearn `classification_report` provides precision, recall, and F1-score for each class, along with macro and weighted averages across all classes.

Class	Precision	Recall	F1-Score	Support
Airplane	0.71	0.78	0.74	1,000
Automobile	0.88	0.71	0.79	1,000
Bird	0.63	0.54	0.58	1,000
Cat	0.47	0.55	0.50	1,000
Deer	0.65	0.61	0.63	1,000
Dog	0.57	0.60	0.59	1,000
Frog	0.78	0.77	0.77	1,000
Horse	0.76	0.75	0.75	1,000
Ship	0.80	0.80	0.80	1,000
Truck	0.73	0.81	0.77	1,000
Macro Average	0.70	0.69	0.69	10,000
Weighted Average	0.70	0.69	0.69	10,000

Table 8.3: Per-class Classification Report (test set)

Precision for a given class measures the fraction of all images predicted as that class that were actually from that class. Recall measures the fraction of all actual images from that class that were correctly identified. F1-Score is the harmonic mean of precision and recall and provides a single balanced measure of per-class performance. The macro average treats all classes equally regardless of support, while the weighted average weights each class's score by its number of test

samples. Since all classes have equal support (1,000 images each), both averages are identical at 0.70 for precision and 0.69 for recall and F1-score.

9. CONCLUSION

This project demonstrated the complete workflow of building, training, and evaluating a Convolutional Neural Network for image classification on the CIFAR-10 dataset. Starting from the raw dataset loaded via TensorFlow's Keras API, the data was normalized and a 7-layer CNN with 167,562 trainable parameters was constructed, trained for 10 epochs, and evaluated on the held-out test set.

The model achieved a test accuracy of 69.13%, which is a reasonable result for a relatively small CNN trained from scratch on CIFAR-10 without data augmentation, regularization (such as dropout or batch normalization), or hyperparameter tuning. The classification report shows that the model performs well on visually distinctive classes such as Automobile (F1: 0.79), Ship (F1: 0.80), Truck (F1: 0.77), and Frog (F1: 0.77), while it struggles with visually ambiguous classes such as Cat (F1: 0.50), Dog (F1: 0.59), and Bird (F1: 0.58). These results are consistent with what is generally observed in the literature for baseline CNNs on CIFAR-10.

The loss and accuracy curves indicate mild overfitting in the final epochs, where training accuracy continues to improve while validation accuracy levels off. The convergence of the test accuracy (69.13%) with the final validation accuracy (69.42%) confirms that training was stable and that the validation accuracy was a reliable indicator of generalization performance.

Several improvements could be explored in future work: adding dropout layers between the Dense layers to reduce overfitting, applying data augmentation techniques (such as random horizontal flipping, rotation, and cropping) to increase the effective size of the training set, increasing the depth of the network, or using transfer learning from a pre-trained model such as VGG or ResNet. The trained model has been saved using `model.save()` in the Keras format for potential future deployment.

10. REFERENCES

TensorFlow. *CIFAR-10 Dataset Documentation*. Available at: https://www.tensorflow.org/api_docs/python/tf/keras/datasets/cifar10 (Accessed: July 2026).

TensorFlow. *Keras Documentation*. Available at: <https://www.tensorflow.org/guide/keras> (Accessed: July 2026).

Scikit-learn Developers. *Scikit-learn Documentation*. Available at: <https://scikit-learn.org/stable/> (Accessed: July 2026).

NumPy Developers. *NumPy Documentation*. Available at: <https://numpy.org/doc/> (Accessed: July 2026).

Matplotlib Development Team. *Matplotlib Documentation*. Available at: <https://matplotlib.org/stable/> (Accessed: July 2026).

Seaborn Developers. *Seaborn Documentation*. Available at: <https://seaborn.pydata.org/> (Accessed: July 2026).

CodeWithHarry. *Python Tutorials and Documentation*. Available at: <https://www.codewithharry.com/> (Accessed: July 2026).

Krish Naik. *Machine Learning and Deep Learning Tutorials*. YouTube Channel. Available at: <https://www.youtube.com/@krishnaik06> (Accessed: July 2026).